Least Squares via Household Reflectors

Jonathan Cushman

November 16, 2025

1 Executive Summary

The following report is about solving linear least squares problems via Householder transformations. The report describes the mathematics behind the problem, a through description of the development of the algorithm, the testing and implementation of the algorithm, and the summarization of the results for each problem tested.

2 Statement of the Problem

In general, the assigned task is to implement the Householder transformation algorithm in order to solve linear least squares problems. Essentially, finding the solution vector to different systems of equations, some with known solutions and some where there is no possible solution. In these cases, we apply the projection theorem in order to approximate a solution to the system. Some specific trends we will be analyzing are square non singular matrices, systems of overdetermined equations, and different basis for regressions. Testing includes an increasing range of problem size with multiple trial runs under random seeds to help generate metrics, and the statistics used to validate our algorithm are working properly upon experimentation.

3 Description of the Mathematics

The goal of the Algorithm is to compute the solution vector $x \in \mathbb{R}^k$ for the least squares problem

$$min_x||Ax - b||_2 \tag{1}$$

where $A \in \mathbb{R}^{n*k}$ with $n \geq k$ and full column rank. If the system is consistent then the solution vector is exactly the solution to $Ax = b$. Otherwise the orthogonal projection of b is cast onto the column space of A.

Then, via a sequence of Householder reflectors, the matrix A is transformed into an upper triangular matrix R. We do this by isolating each active sub vector such that for each column $j = 1, ..., k$

$$a = A(j : n, j) \tag{2}$$

and use this to construct the vector, v that reflects a onto a multiple of the basis vector e_1. We define v such that

$$v = a + sign(a_1) * ||a||_2 * e_1, \quad v \leftarrow v/||v||_2 \tag{3}$$

and the Householder reflector is defined as:

$$H = I - 2vv^T \tag{4}$$

After applying H, all the entries for column j below the diagonal are eliminated. Due to the fact that each H is orthogonal, after applying the Householder reflector, the lease squares problem is preserved while maintaining numerical stability.

Once the reflector is applied to our equation, we yield:

$$A(j : n, j : k) \leftarrow A(j : n, j : k) - 2v(v^T A(j : n, j : k)) \quad and \quad b(j : n) \leftarrow b(j : n) - 2v(v^T b(j : n)) \tag{5}$$

Doing this k times, we now have transformed A into an upper triangular matrix R which is defined as $R = A(1:k, 1:k)$. Along with the transformation of b into $c = b(1:k)$ and $d = b(k+1:n)$.

Now defining the transformed system: $Rx = c$. Additionally note the the sub vector d is the orthogonal component representing the residual:

$$||Ax - b||_2 = ||d||_2 \tag{6}$$

which yields an exaction solution if $d = 0$. In the case this is not true, the least squares solution is produced: $x = min_x||Ax - b||_2$.

By the fact R is upper triangular and non singular, the system $Rx = c$ is solved by back substitution:

$$x_i = 1/R_{ii} * (c_i - \sum_{j=i+1}^{k} R_{ij}x_j), i = k, k-1, ..., 1 \tag{7}$$

Satisfying our inconsistent system of equations

$$Ax = Proj_{\Re(A)}b \tag{8}$$

4 Description of Algorithm and Implementation

The goal of this assignment was to efficiently implement the Householder transformation for solving the linear least squares problem. The following implementation applies a sequence of Householder reflectors that transform the matrix A into the upper triangular Matrix R. We then use this new form for a back-substitution. Each application of the reflectors correlates directly to the active portion of the Matrix and vector associated. This is done to save on storage and efficiency within our algorithm.

For each step j within the algorithm, the active column of A is selected and used to build the Householder vector v that reflects the column onto the first basis vector multiplied by some multiple. Hence the formula: $v = a + sign(a_1) * ||a||_2 * e_1$. Once normalized, the reflector is applied to the active sub-matrix and sub-vector. This process eliminates values below the diagonal in column j. After each reflector is applied, the matrix R is formed, and the transformed vectors yield the vector c, which can be used to compute the solution vector for the system of equations.

Included within the algorithm is a check that verifies whether the diagonal elements of R are numerically unstable for pivoting in the back substitution. After the check is passed, the back solve is computed, where the algorithm produces an exact solution or a minimized solution for an over determined system. The cost of the algorithm in terms of flops is $2nk^2 - 2/3 * k^3$ which yields complexity of $O(2nk^2)$ when the system is overdetermined, and $O(4/3 * n^3)$ when $n = k$.

In General, this algorithm is meant to be numerically robust without storage of unnecessary data points, while remaining safe against ill-conditioned problems.

Listing 1: Algorithm

```
function [x,R] = LLS_House(A,b)

%Input Variable
% - A: Nonsingular Matrix with n x k dimensions
% - b: vector with n x 1 dimensions

%Output Variable
% - x: Solution Vector with k x 1 dimensions

% The solution is either exact OR a projection that numerically
% approximates the true solution onto the Column space of A

% ~ represents an empty variable slot for n
```

```
% efficient to only store columns for computations
[~,k] = size(A);

% Householder Reflector Loop
 for j = 1:k
     % Compute algebraic components of H Reflector
     % Grab active column
     a = A(j:end,j);
     % Initialize and set e1 basis vector
     e1 = zeros(length(a),1);
     e1(1) = 1;
     % Formula Decomposition for vector in Householder Reflector
     v = a + sign(a(1))*norm(a)*e1;
     v = v/ norm(v);

     % Apply Reflectors and Update A and b
     % Note v' represents the transpose of the vector v
     A(j:end,j:end) = A(j:end,j:end) - 2*v*(v'*A(j:end,j:end));
     b(j:end) = b(j:end) - 2*v*(v'*b(j:end));
 end

 % Extraction of the Elements for computations
 % Note that R is Upper triangular
 R = A(1:k,1:k);

 % Check for linear independence
 if min(abs(diag(R))) < eps * max(abs(R(:))) * k %eps is machine precision
    error('Matrix A does not have full column rank');
 end
 c = b(1:k);
% d = b(k+1:end); % Contains residuals for undetermined systems

% Backsolve
x = zeros(k, 1); % Initialize solution vector
for i = k:-1:1
    if i == k
        x(i) = c(i)/R(i,i);
    else
        x(i) = (c(i) - R(i,i+1:k) * x(i+1:k)) / R(i,i);
    end
end
return
```

5 Description of the Experimental Design and Results

The design of this experiment is based on the testing of matrices with varying structures and trends over varying sizes. We then calculate the metrics being used in this experiment: relative solution error, growth factor, conditioning number, and relative residual error. We will test on a number of trial runs per size and examine the mean and max values being output by our algorithm as the size of our matrix grows. The values being used to generate the matrices are random from the open interval $(0, 1)$. All matrix norms used for computations were the standard 2-norm and have the following form:

$$E_{\text{sol}} = \frac{\|x_{true} - x_{comp}\|_2}{\|x_{true}\|_2} \tag{9}$$

$$\gamma = \frac{max|R(:)|}{max|A(:)|} \tag{10}$$

$$E_{\text{res}} = \frac{\| b - Ax_{comp} \|_2}{\|b\|_2} \tag{11}$$

$$K_2(A) = \|A\|_2 \, \|A^{-1}\|_2 \tag{12}$$

the results were summarized using mean and maximum values:

$$\bar{E}_{\text{sol}} = \frac{1}{N} \sum_{i=1}^{N} E_{\text{sol},i}, \qquad E_{\text{sol,max}} = \max_i E_{\text{sol},i}, \tag{13}$$

$$\bar{\gamma} = \frac{1}{N} \sum_{i=1}^{N} \gamma_i, \qquad \gamma_{\max} = \max_i \gamma_i, \tag{14}$$

$$\bar{E}_{\text{res}} = \frac{1}{N} \sum_{i=1}^{N} E_{\text{res},i}, \qquad E_{\text{res,max}} = \max_i E_{\text{res},i}. \tag{15}$$

5.1 General Trends Case 1: Square Non-Singular Matrices

Case 1 involved testing square non-singular systems. From the results of testing, the solver produced results with machine precision accuracy for all test sizes. The metrics produced consistent results with our expected theoretical outputs, with a max solution error underneath the order of 10^{-15}. The residual error was likewise very small and produced consistent results with our expected output. The growth factor stayed between 1 and 2, showing the numerical stability, and the conditioning number slowly increased with the size of the matrix, which is expected for square non singular matrices.

Figure 1: General Trends 1 Metrics, n = k

General Trend Case 1: n = k

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.
5	5	7.19e+00	2.98e-16	4.94e-16	1.03e+00	1.13e+00	1.98e-16	3.39e-16
6	6	9.40e+00	4.70e-16	6.96e-16	1.14e+00	1.31e+00	3.17e-16	5.04e-16
7	7	1.41e+01	4.98e-16	6.34e-16	1.12e+00	1.22e+00	3.63e-16	4.18e-16
8	8	2.18e+01	7.42e-16	1.17e-15	1.17e+00	1.33e+00	3.05e-16	3.37e-16
9	9	4.13e+01	7.28e-16	9.64e-16	1.24e+00	1.39e+00	3.79e-16	5.03e-16
10	10	1.02e+02	2.96e-15	7.14e-15	1.18e+00	1.22e+00	7.47e-16	1.62e-15
11	11	8.70e+01	1.19e-15	3.70e-15	1.34e+00	1.44e+00	5.54e-16	1.13e-15
12	12	6.54e+01	1.45e-15	2.80e-15	1.20e+00	1.47e+00	8.05e-16	1.39e-15
13	13	6.01e+02	8.96e-15	4.00e-14	1.29e+00	1.40e+00	7.38e-15	3.33e-14
14	14	2.64e+02	5.34e-15	1.42e-14	1.44e+00	1.60e+00	3.01e-15	6.05e-15
15	15	9.74e+01	1.44e-15	2.75e-15	1.40e+00	1.61e+00	1.26e-15	4.28e-15
16	16	2.38e+02	2.52e-15	5.07e-15	1.34e+00	1.47e+00	2.18e-15	5.05e-15
17	17	2.22e+02	1.75e-15	3.88e-15	1.47e+00	1.63e+00	1.35e-15	2.40e-15
18	18	1.63e+03	2.67e-14	1.14e-13	1.47e+00	1.70e+00	2.12e-14	1.01e-13
19	19	3.00e+02	4.17e-15	1.28e-14	1.53e+00	1.59e+00	1.37e-15	2.86e-15
20	20	5.89e+02	4.73e-15	1.57e-14	1.64e+00	1.72e+00	5.59e-15	2.38e-14
21	21	4.12e+02	2.98e-15	6.56e-15	1.54e+00	1.70e+00	2.06e-15	5.95e-15
22	22	1.34e+03	2.18e-14	8.76e-14	1.55e+00	1.73e+00	5.24e-15	1.98e-14
23	23	1.36e+02	1.72e-15	3.22e-15	1.62e+00	1.72e+00	8.93e-16	1.43e-15
24	24	4.35e+02	2.43e-15	5.33e-15	1.77e+00	1.87e+00	3.22e-15	8.49e-15
25	25	2.04e+02	1.18e-15	1.62e-15	1.74e+00	2.04e+00	1.55e-15	2.73e-15
26	26	3.42e+02	2.38e-15	4.27e-15	1.72e+00	1.85e+00	1.69e-15	2.82e-15
27	27	6.09e+02	6.77e-15	1.57e-14	1.75e+00	1.86e+00	2.50e-15	5.01e-15
28	28	3.69e+02	2.74e-15	6.34e-15	1.64e+00	1.84e+00	1.76e-15	4.73e-15
29	29	6.94e+02	5.43e-15	1.63e-14	1.78e+00	1.89e+00	1.31e-15	2.13e-15
30	30	7.11e+02	3.93e-15	6.95e-15	1.80e+00	1.90e+00	2.21e-15	3.27e-15

5.2 General Trends Case 2: Overdetermined systems with $b \in \Re(A)$

Case 2 involved testing rectangular overdetermined systems where $b \in \Re(A)$. From the results, we see that both our solution error and residual error were near machine precision, indicating that $b \in \Re(A)$. Additionally, we notice that the growth factor remains stable and that the condition number slowly grows, similar to case 1.

Figure 2: General Trends 2 Metrics, $n \geq k$ s.t. $b \in \Re(A)$

General Trend Case 2: n > k s.t. b in R(A)

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.
5	4	6.12e+00	5.15e-16	1.05e-15	1.08e+00	1.22e+00	2.23e-16	3.04e-16
6	5	6.63e+00	5.10e-16	7.68e-16	1.15e+00	1.25e+00	1.60e-16	3.30e-16
7	6	1.20e+01	9.89e-16	1.91e-15	1.18e+00	1.43e+00	2.05e-16	3.45e-16
8	7	1.17e+01	7.29e-16	1.02e-15	1.18e+00	1.29e+00	2.05e-16	3.35e-16
9	8	2.38e+01	9.42e-16	1.60e-15	1.21e+00	1.36e+00	2.26e-16	4.16e-16
10	9	2.32e+01	2.25e-15	6.05e-15	1.25e+00	1.36e+00	2.59e-16	4.32e-16
11	10	2.92e+01	1.88e-15	3.28e-15	1.24e+00	1.37e+00	2.42e-16	3.88e-16
12	11	3.57e+01	1.63e-15	3.41e-15	1.27e+00	1.42e+00	1.95e-16	2.54e-16
13	12	3.07e+01	1.92e-15	3.92e-15	1.23e+00	1.38e+00	1.95e-16	2.63e-16
14	13	5.44e+01	1.38e-15	2.42e-15	1.31e+00	1.47e+00	2.16e-16	2.63e-16
15	14	4.98e+01	2.17e-15	3.34e-15	1.39e+00	1.60e+00	2.14e-16	3.62e-16
16	15	5.60e+01	2.02e-15	4.16e-15	1.42e+00	1.56e+00	2.34e-16	3.10e-16
17	16	9.52e+01	3.94e-15	6.90e-15	1.38e+00	1.49e+00	2.11e-16	3.10e-16
18	17	1.06e+02	3.04e-15	5.59e-15	1.48e+00	1.67e+00	2.34e-16	3.86e-16
19	18	6.93e+01	2.36e-15	4.66e-15	1.46e+00	1.63e+00	2.19e-16	3.32e-16
20	19	1.62e+02	8.17e-15	2.60e-14	1.55e+00	1.68e+00	2.45e-16	3.72e-16
21	20	2.33e+02	1.22e-14	4.55e-14	1.61e+00	1.81e+00	2.98e-16	4.19e-16
22	21	1.59e+02	5.02e-15	1.23e-14	1.58e+00	1.72e+00	2.38e-16	3.71e-16
23	22	1.34e+02	4.72e-15	6.64e-15	1.64e+00	1.88e+00	2.73e-16	3.52e-16
24	23	1.59e+02	5.07e-15	1.06e-14	1.61e+00	1.79e+00	3.12e-16	4.91e-16
25	24	1.37e+02	6.32e-15	1.33e-14	1.71e+00	1.79e+00	3.15e-16	3.51e-16
26	25	2.16e+02	7.45e-15	1.43e-14	1.77e+00	2.11e+00	2.05e-16	2.41e-16
27	26	1.71e+02	4.36e-15	7.96e-15	1.74e+00	1.90e+00	2.89e-16	4.07e-16
28	27	2.83e+02	5.17e-15	1.11e-14	1.72e+00	1.80e+00	2.08e-16	2.59e-16
29	28	2.93e+02	1.09e-14	2.30e-14	1.67e+00	1.85e+00	2.67e-16	4.25e-16
30	29	3.31e+02	9.59e-15	2.49e-14	1.70e+00	1.89e+00	2.78e-16	3.29e-16

5.3 General Trends Case 3: Overdetermined systems with $b \notin \Re(A)$

Wrapping up testing for the general trends, case 3 is similar to that of case 2, expect the vector $b \notin \Re(A)$. From the results, the metrics are very similar to those of case number 2, with the main difference being in the residual errors. In Case 3, the residual error is much closer to 10^{-2}, indicating the vector $b \notin \Re(A)$. While the low solution error indicates that our algorithm is working well. One additional thing to note for the results is that the stability of the algorithm is independent of the consistency of the system.

Figure 3: General Trends 3 Metrics, $n \geq k$ s.t. $b \notin \Re(A)$

General Trend Case 3: n > k s.t. b not in R(A)

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.
5	4	5.63e+00	4.66e-16	1.04e-15	1.08e+00	1.20e+00	8.42e-02	1.79e-01
6	5	8.80e+00	7.82e-16	1.57e-15	1.11e+00	1.27e+00	3.24e-02	6.60e-02
7	6	7.98e+00	7.66e-16	1.10e-15	1.18e+00	1.33e+00	3.57e-02	7.01e-02
8	7	1.57e+01	9.34e-16	1.40e-15	1.19e+00	1.29e+00	3.98e-02	6.99e-02
9	8	2.19e+01	1.05e-15	2.00e-15	1.15e+00	1.25e+00	2.46e-02	3.38e-02
10	9	1.71e+01	1.27e-15	2.02e-15	1.26e+00	1.44e+00	2.27e-02	4.88e-02
11	10	3.92e+01	1.74e-15	3.19e-15	1.31e+00	1.47e+00	2.79e-02	5.40e-02
12	11	2.66e+01	1.33e-15	1.89e-15	1.20e+00	1.25e+00	1.62e-02	3.13e-02
13	12	2.65e+01	1.31e-15	2.49e-15	1.36e+00	1.49e+00	1.02e-02	2.26e-02
14	13	5.59e+01	1.12e-15	2.09e-15	1.31e+00	1.42e+00	1.38e-02	1.99e-02
15	14	3.59e+01	1.56e-15	2.32e-15	1.26e+00	1.39e+00	9.37e-03	1.47e-02
16	15	1.96e+02	4.30e-14	2.09e-13	1.51e+00	1.63e+00	1.63e-02	2.79e-02
17	16	5.66e+01	1.38e-15	1.75e-15	1.60e+00	1.73e+00	9.26e-03	2.61e-02
18	17	1.60e+02	2.12e-15	3.17e-15	1.46e+00	1.56e+00	3.74e-03	7.68e-03
19	18	7.72e+01	2.40e-15	3.56e-15	1.48e+00	1.66e+00	1.48e-02	3.26e-02
20	19	1.01e+02	2.83e-15	5.73e-15	1.53e+00	1.71e+00	6.44e-03	9.53e-03
21	20	8.40e+01	3.03e-15	5.42e-15	1.47e+00	1.61e+00	9.07e-03	1.45e-02
22	21	1.09e+02	2.18e-15	3.36e-15	1.52e+00	1.59e+00	5.78e-03	8.31e-03
23	22	2.72e+02	1.40e-14	5.28e-14	1.61e+00	2.00e+00	5.32e-03	9.85e-03
24	23	2.90e+02	4.47e-15	1.00e-14	1.54e+00	1.59e+00	6.80e-03	1.41e-02
25	24	2.83e+02	9.12e-15	2.74e-14	1.64e+00	1.78e+00	6.53e-03	1.07e-02
26	25	1.26e+02	1.97e-15	3.97e-15	1.67e+00	1.87e+00	1.16e-02	1.81e-02
27	26	1.81e+02	3.06e-15	4.58e-15	1.72e+00	1.98e+00	6.15e-03	8.73e-03
28	27	1.68e+02	3.80e-15	5.23e-15	1.72e+00	1.81e+00	8.35e-03	1.77e-02
29	28	1.60e+02	5.01e-15	1.19e-14	1.69e+00	1.78e+00	7.36e-03	9.25e-03
30	29	1.90e+02	5.05e-15	6.69e-15	1.86e+00	1.96e+00	2.44e-03	4.93e-03

5.4 Required Problem 1: Growth Factor Matrix

For the next set of problems, we start by revisiting the growth factor matrix from program 3. From the results, we notice that our algorithm is working properly with good low metrics all around. The more interesting detail to notice is the growth factor associating with the Householder transform in comparison to the LU Factorization from Program 3. Using the result from the 3rd assignment, the growth factor exploded when using the no pivoting and partial pivoting factorization methods. The opposite is true for the Householder algorithm, which yields a linear growth rather than an exponential one.

Figure 4: Growth Factor Matrix Metrics via Householder

Required Problem 1: Growth Factor Matrix

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.
5	5	2.22e+00	5.82e-16	8.86e-16	2.24e+00	2.24e+00	2.04e-16	2.34e-16
6	6	2.65e+00	6.01e-16	8.71e-16	2.45e+00	2.45e+00	2.18e-16	2.49e-16
7	7	3.08e+00	1.98e-15	3.43e-15	2.65e+00	2.65e+00	2.42e-16	2.76e-16
8	8	3.51e+00	2.23e-15	2.72e-15	2.83e+00	2.83e+00	2.19e-16	3.14e-16
9	9	3.95e+00	6.73e-15	9.73e-15	3.00e+00	3.00e+00	2.25e-16	2.66e-16
10	10	4.38e+00	1.28e-14	1.91e-14	3.16e+00	3.16e+00	3.02e-16	3.70e-16
11	11	4.83e+00	1.45e-14	2.34e-14	3.32e+00	3.32e+00	2.43e-16	2.73e-16
12	12	5.27e+00	3.95e-14	6.23e-14	3.46e+00	3.46e+00	2.38e-16	3.40e-16
13	13	5.71e+00	9.07e-14	1.60e-13	3.61e+00	3.61e+00	2.66e-16	4.10e-16
14	14	6.16e+00	2.37e-13	5.79e-13	3.74e+00	3.74e+00	3.02e-16	3.80e-16
15	15	6.60e+00	3.81e-13	7.35e-13	3.87e+00	3.87e+00	2.59e-16	3.64e-16
16	16	7.05e+00	6.18e-13	8.45e-13	4.00e+00	4.00e+00	3.73e-16	5.21e-16
17	17	7.49e+00	1.83e-12	2.76e-12	4.12e+00	4.12e+00	3.62e-16	4.71e-16
18	18	7.94e+00	2.80e-12	5.97e-12	4.24e+00	4.24e+00	2.61e-16	3.27e-16
19	19	8.39e+00	6.38e-12	1.37e-11	4.36e+00	4.36e+00	3.61e-16	5.04e-16
20	20	8.83e+00	1.44e-11	2.26e-11	4.47e+00	4.47e+00	2.98e-16	4.33e-16
21	21	9.28e+00	1.77e-11	4.55e-11	4.58e+00	4.58e+00	2.85e-16	3.15e-16
22	22	9.73e+00	5.33e-11	1.05e-10	4.69e+00	4.69e+00	2.91e-16	3.40e-16
23	23	1.02e+01	4.64e-11	1.01e-10	4.80e+00	4.80e+00	3.83e-16	5.01e-16
24	24	1.06e+01	1.22e-10	1.65e-10	4.90e+00	4.90e+00	3.93e-16	4.46e-16
25	25	1.11e+01	4.30e-10	8.97e-10	5.00e+00	5.00e+00	3.30e-16	5.09e-16
26	26	1.15e+01	4.52e-10	7.97e-10	5.10e+00	5.10e+00	3.62e-16	5.90e-16
27	27	1.20e+01	1.39e-09	2.48e-09	5.20e+00	5.20e+00	3.29e-16	4.18e-16
28	28	1.24e+01	2.31e-09	5.70e-09	5.29e+00	5.29e+00	3.61e-16	4.80e-16
29	29	1.29e+01	5.38e-09	1.52e-08	5.39e+00	5.39e+00	3.27e-16	4.37e-16
30	30	1.33e+01	7.07e-09	1.33e-08	5.48e+00	5.48e+00	3.66e-16	6.35e-16

Figure 5: Growth Factor Matrix Metrics via LU (Program 3)

Task 1.7: Growth Factor Matrix

n	cond(A)	piv	mean_err_fac	max_err_fac	mean_gamma	max_gamma	mean_resid	max_resid
5	2.22e+00	1	0.00e+00	0.00e+00	11.26	11.26	0.00e+00	0.00e+00
5	2.22e+00	2	0.00e+00	0.00e+00	11.26	11.26	0.00e+00	0.00e+00
5	2.22e+00	3	0.00e+00	0.00e+00	2.97	2.97	0.00e+00	0.00e+00
10	4.38e+00	1	0.00e+00	0.00e+00	190.40	190.40	0.00e+00	0.00e+00
10	4.38e+00	2	0.00e+00	0.00e+00	190.40	190.40	0.00e+00	0.00e+00
10	4.38e+00	3	0.00e+00	0.00e+00	3.06	3.06	0.00e+00	0.00e+00
15	6.60e+00	1	0.00e+00	0.00e+00	4052.70	4052.70	0.00e+00	0.00e+00
15	6.60e+00	2	0.00e+00	0.00e+00	4052.70	4052.70	0.00e+00	0.00e+00
15	6.60e+00	3	0.00e+00	0.00e+00	3.05	3.05	0.00e+00	0.00e+00
20	8.83e+00	1	0.00e+00	0.00e+00	96912.46	96912.46	0.00e+00	0.00e+00
20	8.83e+00	2	0.00e+00	0.00e+00	96912.46	96912.46	0.00e+00	0.00e+00
20	8.83e+00	3	0.00e+00	0.00e+00	3.05	3.05	0.00e+00	0.00e+00
25	1.11e+01	1	0.00e+00	0.00e+00	2473963.22	2473963.22	0.00e+00	0.00e+00
25	1.11e+01	2	0.00e+00	0.00e+00	2473963.22	2473963.22	0.00e+00	0.00e+00
25	1.11e+01	3	0.00e+00	0.00e+00	3.04	3.04	0.00e+00	0.00e+00
30	1.33e+01	1	0.00e+00	0.00e+00	65830866.53	65830866.53	0.00e+00	0.00e+00
30	1.33e+01	2	0.00e+00	0.00e+00	65830866.53	65830866.53	0.00e+00	0.00e+00
30	1.33e+01	3	0.00e+00	0.00e+00	3.04	3.04	0.00e+00	0.00e+00
35	1.56e+01	1	0.00e+00	0.00e+00	1802644652.01	1802644652.01	0.00e+00	0.00e+00
35	1.56e+01	2	0.00e+00	0.00e+00	1802644652.01	1802644652.01	0.00e+00	0.00e+00
35	1.56e+01	3	0.00e+00	0.00e+00	3.03	3.03	0.00e+00	0.00e+00
40	1.78e+01	1	0.00e+00	0.00e+00	50407485251.57	50407485251.57	0.00e+00	0.00e+00
40	1.78e+01	2	0.00e+00	0.00e+00	50407485251.57	50407485251.57	0.00e+00	0.00e+00
40	1.78e+01	3	0.00e+00	0.00e+00	3.03	3.03	0.00e+00	0.00e+00
45	2.01e+01	1	0.00e+00	0.00e+00	1432280894243.95	1432280894243.95	0.00e+00	0.00e+00
45	2.01e+01	2	0.00e+00	0.00e+00	1432280894243.95	1432280894243.95	0.00e+00	0.00e+00
45	2.01e+01	3	0.00e+00	0.00e+00	3.03	3.03	0.00e+00	0.00e+00
50	2.23e+01	1	0.00e+00	0.00e+00	41213315627127.97	41213315627127.97	0.00e+00	0.00e+00
50	2.23e+01	2	0.00e+00	0.00e+00	41213315627127.97	41213315627127.97	0.00e+00	0.00e+00
50	2.23e+01	3	0.00e+00	0.00e+00	3.02	3.02	0.00e+00	0.00e+00

5.5 Required Problem 2: Arithmetic Mean

The 2nd required problem from this assignment was solving the arithmetic mean linear least squares system. In this case, the results yielded extremely low solution errors along with perfect conditioning to accompany the result. This implies that our algorithm is correctly computing the mean. Another thing to notice from the results is the somewhat large residual values. This is to be expected from our theoretical view, as the residual is the representation of the deviation from the mean, thus showing that this is not an error but rather variability from the random data. Lastly, we note a linearly increasing growth factor similar to before.

Figure 6: Arithmetic Mean Metrics

Required Problem 2: Arithmetic Mean

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.
5	1	1.00e+00	1.37e-16	2.95e-16	2.24e+00	2.24e+00	4.70e-01	5.94e-01
6	1	1.00e+00	8.89e-17	2.12e-16	2.45e+00	2.45e+00	4.88e-01	7.57e-01
7	1	1.00e+00	1.11e-16	2.01e-16	2.65e+00	2.65e+00	4.47e-01	6.20e-01
8	1	1.00e+00	1.70e-16	3.42e-16	2.83e+00	2.83e+00	4.99e-01	6.69e-01
9	1	1.00e+00	6.98e-17	2.01e-16	3.00e+00	3.00e+00	3.88e-01	4.74e-01
10	1	1.00e+00	1.91e-16	4.26e-16	3.16e+00	3.16e+00	4.85e-01	5.93e-01
11	1	1.00e+00	1.42e-16	3.78e-16	3.32e+00	3.32e+00	4.20e-01	4.88e-01
12	1	1.00e+00	1.22e-16	2.00e-16	3.46e+00	3.46e+00	4.64e-01	5.47e-01
13	1	1.00e+00	4.79e-17	2.39e-16	3.61e+00	3.61e+00	4.24e-01	5.50e-01
14	1	1.00e+00	2.50e-16	5.54e-16	3.74e+00	3.74e+00	5.06e-01	5.82e-01
15	1	1.00e+00	1.11e-16	1.92e-16	3.87e+00	3.87e+00	4.30e-01	5.55e-01
16	1	1.00e+00	1.75e-16	3.82e-16	4.00e+00	4.00e+00	5.08e-01	5.55e-01
17	1	1.00e+00	1.28e-16	2.25e-16	4.12e+00	4.12e+00	4.79e-01	5.55e-01
18	1	1.00e+00	4.22e-16	7.42e-16	4.24e+00	4.24e+00	4.39e-01	5.49e-01
19	1	1.00e+00	3.18e-16	6.06e-16	4.36e+00	4.36e+00	4.79e-01	5.74e-01
20	1	1.00e+00	3.39e-16	6.32e-16	4.47e+00	4.47e+00	4.62e-01	5.95e-01
21	1	1.00e+00	2.26e-16	6.57e-16	4.58e+00	4.58e+00	5.28e-01	5.78e-01
22	1	1.00e+00	2.71e-16	5.92e-16	4.69e+00	4.69e+00	5.53e-01	6.49e-01
23	1	1.00e+00	2.56e-16	5.76e-16	4.80e+00	4.80e+00	4.79e-01	6.24e-01
24	1	1.00e+00	2.24e-16	4.52e-16	4.90e+00	4.90e+00	5.06e-01	5.91e-01
25	1	1.00e+00	3.49e-16	4.37e-16	5.00e+00	5.00e+00	4.80e-01	5.08e-01
26	1	1.00e+00	1.78e-16	3.48e-16	5.10e+00	5.10e+00	5.17e-01	5.45e-01
27	1	1.00e+00	2.84e-16	6.42e-16	5.20e+00	5.20e+00	5.09e-01	5.83e-01
28	1	1.00e+00	1.60e-16	3.91e-16	5.29e+00	5.29e+00	5.10e-01	5.65e-01
29	1	1.00e+00	2.11e-16	3.85e-16	5.39e+00	5.39e+00	4.91e-01	5.54e-01
30	1	1.00e+00	4.10e-16	6.44e-16	5.48e+00	5.48e+00	4.81e-01	5.38e-01

5.6 Required Problem 3(a): Monomial Basis

The next set of problems tested is for the monomial bases. The first noticeable finding from this test was the solution errors being near machine precision accuracy, indicating that our algorithm is working as expected. The next thing to notice is the residuals and the relationship it has with the value of d. The higher degree we are able to use, the better approximation we are able to get. Reinforcing the results of having a smaller residual value when d is higher. Conversely, the relationship between the conditioning of the matrix seems to get worse the higher the value of d. Another interesting result from testing was the evaluation of the Gram Matrix. Looking at the structure that emerges as d increases in size, a pattern begins to appear where the elements on the diagonal are mirrored across their respective anti diagonals, except on the top left and bottom right corners. This leads to problems as there is an increasing level of correlation between the off diagonal and diagonal elements. This can lead to severe growth in the conditioning number, leading to a more preferable case, which is analyzed next.

Figure 7: Monomial Basis Matrix Metrics

Required Problem 3: Monomial Basis for d = 1,2,3

n	k	cond(A)	Mean S.E.	Max S.E.	Mean G.F.	Max G.F	Mean R.E.	Max R.E.	
5	1	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
5	2	1.41e+00	4.75e-16	7.65e-16	2.24e+00	2.24e+00	4.39e-01	6.15e-01	<-- d=1
5	3	3.08e+00	5.72e-16	8.59e-16	2.24e+00	2.24e+00	3.85e-01	5.22e-01	<-- d=2
5	4	7.10e+00	8.94e-16	1.29e-15	2.24e+00	2.24e+00	2.18e-01	4.83e-01	<-- d=3
6	1	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
6	2	1.46e+00	3.51e-16	4.80e-16	2.45e+00	2.45e+00	4.45e-01	6.25e-01	<-- d=1
6	3	3.13e+00	3.22e-16	5.08e-16	2.45e+00	2.45e+00	3.70e-01	6.07e-01	<-- d=2
6	4	6.91e+00	9.73e-16	1.80e-15	2.45e+00	2.45e+00	3.27e-01	5.91e-01	<-- d=3
7	1	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
7	2	1.50e+00	1.36e-16	5.69e-16	2.65e+00	2.65e+00	4.46e-01	5.39e-01	<-- d=1
7	3	3.18e+00	3.73e-16	7.09e-16	2.65e+00	2.65e+00	4.13e-01	5.33e-01	<-- d=2
7	4	6.89e+00	5.19e-16	8.92e-16	2.65e+00	2.65e+00	3.37e-01	4.62e-01	<-- d=3
8	1	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
8	2	1.53e+00	1.95e-16	3.01e-16	2.83e+00	2.83e+00	4.38e-01	5.29e-01	<-- d=1
8	3	3.23e+00	3.49e-16	6.45e-16	2.83e+00	2.83e+00	3.87e-01	4.83e-01	<-- d=2
8	4	6.93e+00	1.00e-15	2.39e-15	2.83e+00	2.83e+00	3.51e-01	4.83e-01	<-- d=3
9	1	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	
9	2	1.55e+00	3.27e-16	5.75e-16	3.00e+00	3.00e+00	5.08e-01	5.64e-01	<-- d=1
9	3	3.27e+00	3.98e-16	7.53e-16	3.00e+00	3.00e+00	4.76e-01	5.24e-01	<-- d=2
9	4	6.98e+00	5.93e-16	9.86e-16	3.00e+00	3.00e+00	4.49e-01	4.84e-01	<-- d=3

Figure 8: Monomial Gram Matrix

Gram matrix for Monomial basis (n=10, d=1):

10.0000	0
0	4.0741

Gram matrix for Monomial basis (n=10, d=2):

10.0000	0	4.0741
0	4.0741	0
4.0741	0	2.9474

Gram matrix for Monomial basis (n=10, d=3):

10.0000	0.0000	4.0741	0
0.0000	4.0741	0	2.9474
4.0741	0	2.9474	0
0	2.9474	0	2.5043

5.7 Required Problem 3(b): Chebyshev Basis

The last case is used to evaluate the performance of the linear least squares solution using the Chebyshev polynomial bases. The algorithm was tested for varying degrees of $d = 1, 2, 3$ along two types of interpolation points. The first was a uniformly spaced separation of notes and the Chebyshev root nodes. Looking into the metrics, the conditioning of uniform points yielded a higher condition number over all degrees, while the root points improved the conditioning. Both the solution error and residual error were lower when using the Chebyshev root points as well. Note that, similarly to case 5, the residual error is much higher than the solution error. This is to be expected as we are projecting our solution and limited by our models ability to fit the data. Another thing to notice from testing this case is the associated Gram matrix for both points. Similar to the monomial case, the structure of the Gram matrix continues with the expectation that the off-diagonal elements are much lower than the elements on the main diagonal. This is an improvement to before, where our matrix was not diagonally dominant. Going even further, the Gram matrix associated with the Chebyshev root points the matrix is nearly diagonal. This allows for a better-scaled matrix with increased numerical stability.

Figure 9: Chebyshev Basis Matrix Metrics

```
Required Problem 4: Chebyshev Basis for d = 1,2,3
  n |  k  | cond(A)  | Mean S.E.| Max S.E. | Mean G.F.| Max G.F  | Mean R.E.| Max R.E.
----|-----|----------|----------|----------|----------|----------|----------|----------

  --- UNIFORM POINTS ---
  5 |   2 | 1.41e+00 | 5.09e-16 | 6.79e-16 | 2.24e+00 | 2.24e+00 | 2.90e-01 | 3.59e-01   <-- d=1 (uniform)
  5 |   3 | 1.41e+00 | 4.90e-16 | 7.12e-16 | 2.24e+00 | 2.24e+00 | 2.57e-01 | 3.50e-01   <-- d=2 (uniform)
  5 |   4 | 1.58e+00 | 5.09e-16 | 7.58e-16 | 2.24e+00 | 2.24e+00 | 2.45e-01 | 3.50e-01   <-- d=3 (uniform)

  --- CHEBYSHEV ROOT POINTS ---
  2 |   2 | 2.83e-01 | 4.05e-17 | 2.02e-16 | 2.83e-01 | 1.41e+00 | 2.32e-17 | 1.16e-16   <-- d=1 (Cheb roots)
  3 |   3 | 2.83e-01 | 3.52e-17 | 1.76e-16 | 3.46e-01 | 1.73e+00 | 2.00e-17 | 9.98e-17   <-- d=2 (Cheb roots)
  4 |   4 | 2.83e-01 | 8.71e-17 | 4.36e-16 | 4.00e-01 | 2.00e+00 | 7.60e-17 | 3.80e-16   <-- d=3 (Cheb roots)
  5 |   1 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00
  5 |   2 | 1.41e+00 | 5.09e-16 | 6.79e-16 | 2.24e+00 | 2.24e+00 | 2.90e-01 | 3.59e-01   <-- d=1
  5 |   3 | 1.41e+00 | 4.90e-16 | 7.12e-16 | 2.24e+00 | 2.24e+00 | 2.57e-01 | 3.50e-01   <-- d=2
  5 |   4 | 1.58e+00 | 5.09e-16 | 7.58e-16 | 2.24e+00 | 2.24e+00 | 2.45e-01 | 3.50e-01   <-- d=3

  --- UNIFORM POINTS ---
  6 |   2 | 1.46e+00 | 2.13e-16 | 5.09e-16 | 2.45e+00 | 2.45e+00 | 3.68e-01 | 4.22e-01   <-- d=1 (uniform)
  6 |   3 | 1.47e+00 | 2.50e-16 | 5.63e-16 | 2.45e+00 | 2.45e+00 | 3.41e-01 | 3.86e-01   <-- d=2 (uniform)
  6 |   4 | 1.54e+00 | 2.33e-16 | 3.67e-16 | 2.45e+00 | 2.45e+00 | 1.98e-01 | 2.70e-01   <-- d=3 (uniform)

  --- CHEBYSHEV ROOT POINTS ---
  2 |   2 | 2.83e-01 | 7.84e-18 | 3.92e-17 | 2.83e-01 | 1.41e+00 | 2.24e-17 | 1.12e-16   <-- d=1 (Cheb roots)
  3 |   3 | 2.83e-01 | 4.38e-18 | 2.19e-17 | 3.46e-01 | 1.73e+00 | 2.19e-17 | 1.09e-16   <-- d=2 (Cheb roots)
  4 |   4 | 2.83e-01 | 3.39e-17 | 1.69e-16 | 4.00e-01 | 2.00e+00 | 3.08e-17 | 1.54e-16   <-- d=3 (Cheb roots)
  6 |   1 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00 | 0.00e+00
  6 |   2 | 1.46e+00 | 2.13e-16 | 5.09e-16 | 2.45e+00 | 2.45e+00 | 3.68e-01 | 4.22e-01   <-- d=1
  6 |   3 | 1.47e+00 | 2.50e-16 | 5.63e-16 | 2.45e+00 | 2.45e+00 | 3.41e-01 | 3.86e-01   <-- d=2
  6 |   4 | 1.54e+00 | 2.33e-16 | 3.67e-16 | 2.45e+00 | 2.45e+00 | 1.98e-01 | 2.70e-01   <-- d=3

  --- UNIFORM POINTS ---
  7 |   2 | 1.50e+00 | 2.57e-16 | 4.12e-16 | 2.65e+00 | 2.65e+00 | 5.17e-01 | 6.67e-01   <-- d=1 (uniform)
  7 |   3 | 1.52e+00 | 3.59e-16 | 5.38e-16 | 2.65e+00 | 2.65e+00 | 4.44e-01 | 6.10e-01   <-- d=2 (uniform)
  7 |   4 | 1.54e+00 | 3.80e-16 | 4.62e-16 | 2.65e+00 | 2.65e+00 | 3.96e-01 | 5.20e-01   <-- d=3 (uniform)
```

Figure 10: Chebyshev Gram Matrix

```
Gram matrix for Chebyshev basis with uniform points (n=10, d=1):
 10.0000         0
       0    4.0741


Gram matrix for Chebyshev basis with uniform points (n=10, d=2):
 10.0000         0   -1.8519
       0    4.0741         0
 -1.8519         0    5.4934


Gram matrix for Chebyshev basis with uniform points (n=10, d=3):
 10.0000    0.0000   -1.8519    0.0000
  0.0000    4.0741         0   -0.4326
 -1.8519         0    5.4934    0.0000
  0.0000   -0.4326    0.0000    5.9975


Gram matrix for Chebyshev basis with root points (n=10, d=1):
  2.0000    0.0000
  0.0000    1.0000


Gram matrix for Chebyshev basis with root points (n=10, d=2):
  3.0000    0.0000    0.0000
  0.0000    1.5000   -0.0000
  0.0000   -0.0000    1.5000


Gram matrix for Chebyshev basis with root points (n=10, d=3):
  4.0000    0.0000   -0.0000   -0.0000
  0.0000    2.0000         0   -0.0000
 -0.0000         0    2.0000         0
 -0.0000   -0.0000         0    2.0000
```

5.8 Polynomial Comparison: Monomial vs Chebyshev

The final step for testing was comparing the monomial and Chebyshev bases alongside one another to asses the numerical behavior of the different bases in least squares regression. The test was set up by fitting a quadratic model to noisy data sampled from a perturbed vector over uniformly spaced points between -1 and 1. From testing, both models produce almost identical solutions with a difference of approximately 10^{-16}. The main point to address is the conditioning of the matrices. The Chebyshev Matrix demonstrated higher numerical stability, suggesting that it would be a better choice for higher degree fits.

6 Conclusion

The purpose of this assignment was to showcase the solution to the linear least squares problem via Householder transformations. Evidenced through testing and validation of the algorithm working as expected on various test cases. The algorithm is working properly and outputs data that matches the theoretical outputs. The Householder method has many advantages over previous algorithms that have been tested and have been shown to have tremendous potential for applications in higher-degree regression modeling. Overall, the algorithm preformed well among the various test cases and produced results that mirrored the expected outputs.

7 Program Files

The Program Includes the following files: LLSHouse.m, Program4TestDriver.m, and Chebyshev.m

Note: All programs run at once